


# Windows : réflexion et frontières — A-EPI-004

## Réflexion, frontières et correctifs incomplets

Ted “Tedsig42” Kouhouenou ted.sig42@tutamail.com

*Summary: Module avancé : modéliser la chaîne d'authentification Windows comme une machine à états, puis analyser une famille de CVE récentes qui partagent une racine de réflexion, de contournement de frontière ou de chargement privilégié. Le laboratoire est un AD simulé, jamais une cible réelle.*

*Prerequisites : A-EPI-001, bases d'Active Directory, NTLM, Kerberos, SMB*

*Version: 1.0*

# Contents


**I Instructions** **2**

**II Instructions Communes** **5**

II.1 Structure du Dépôt . . . . . 5

II.2 Qu'est-ce qu'une Preuve de Concept Valide . . . . . 5

II.3 Processus de Soumission . . . . . 6

II.4 Règle des Fichiers Supplémentaires Interdits . . . . . 7

II.5 Format du Rapport . . . . . 7

II.6 Cibles Autorisées Uniquement . . . . . 8

II.7 Citez Toutes Vos Sources . . . . . 8

**III Instructions sur l'IA** **10**

III.1 Aperçu de la Politique . . . . . 10

III.2 Utilisations Autorisées . . . . . 10

III.3 Utilisations Interdites . . . . . 11

III.4 Responsabilité Personnelle . . . . . 11

III.5 Citation Obligatoire de l'IA . . . . . 11

III.6 Évaluation par les Pairs et l'IA . . . . . 12

III.7 Rappel Éthique . . . . . 13

**IV Avant-propos** **14**

**V Aperçu du module** **15**

**VI Exercise 00** **16**

**VII Exercise 01** **17**

**VIII Exercise 02** **18**

**IX Exercise 03** **19**

**X Exercise 04** **21**

**XI Rendu et évaluation** **23**

**XII Ressources par exercice** **24**

**XIII Sources** **26**

# Chapter I

# Instructions

- Trois jours.
- Aucun test sur un domaine de production ou un domaine tiers.
- Le laboratoire est un environnement Active Directory simulé, isolé, avec des comptes jetables.
- Aucune coercition d'authentification, aucun relais réel, aucun outil offensif contre une cible non autorisée.
- Les CVE de 2026 doivent être traitées comme des cas d'étude : vérifiez chaque affirmation dans les avis publics.
- Dark Elevator et CVE-2026-66804 sont des cas de logique et de correctif incomplet : aucune reproduction d'exploit n'est demandée.

## Format du rendu — en-tête obligatoire

Tout fichier rendu dans ce module doit commencer par l'en-tête ci-dessous, **adapté à la syntaxe de commentaire du langage** utilisé.

### Modèle générique

```
/**
 * Module: <code module>
 * Exercise: <numéro exo>
 * File: <nom du fichier>
 * Author: <votre login JeuneCrack>
 *
 * Description: <une phrase, pas de roman>
 *
 * Return: <voir ci-dessous>
 */
```

### Login

Votre identifiant **JeuneCrack** (ex: `tedsig42`). Pour un rendu en binôme : `login1/login2`. Ajoutez un handle Matrix après le login si vous le souhaitez (ex:

`tedsig42 @tedsig42`).

### Return — selon le type de fichier

| Type de fichier | Valeur de Return |
|---|---|
| C (`.c`) | `return (0);` (succès), `return (-1);` (erreur), `return (valeur);` (résultat calculé) |
| Python (`.py`) | `0` (succès), `1` (erreur), `N/A` (analyse, script sans exit) |
| Shell (`.sh`) | `exit 0` (succès), `exit 1` (erreur) |
| Markdown (`.md`) | `N/A` — les fichiers d'analyse n'ont pas de valeur de retour |
| Exploit C/Python | `shell` (shell obtenu), `flag: <flag>` (flag capturé), `proof: <hash>` (preuve de concept) |
| Binaire (`.bin`) | `N/A` — pas d'en-tête pour les fichiers binaires |

### Exemples par langage

**C (`.c`) :**

```c
/**
 * Module: F-EXP-001
 * Exercise: ex02
 * File: exploit.c
 * Author: tedsig42
 *
 * Description: Buffer overflow with ret2libc
 *
 * Return: 0
 */
```

**Python (`.py`) :**

```python
# Module: F-WEB-001
# Exercise: ex05
# File: ssrf-exploit.py
# Author: tedsig42
#
# Description: Blind SSRF bypass via redirect
#
# Return: 0
```

**Shell (`.sh`) :**

```shell
# Module: F-SUP-001
# Exercise: ex03
# File: dependency-confusion.sh
# Author: tedsig42
#
# Description: Automated dependency confusion test
```

```
#
# Return: exit 0
```

**Markdown (`.md`)** — analyse, rapport :

```
<!--
Module: M-SRC-001
Exercise: ex04
File: audit-report.md
Author: tedsig42

Description: Code audit of vulnerable-httpd

Return: N/A
-->
```

# Chapter II

# Instructions Communes

## II.1 Structure du Dépôt

Votre dépôt Git doit respecter la convention de nommage suivante pour tous les modules soumis. L'outil d'évaluation (`pendragon`) se base sur cette structure pour localiser et tester votre travail.

```
<login>/
+-- <MODULE-ID>-<slug>/         # ex. M-BLS-001-business-logic/
|   +-- <turnindir>/            # ex. ex00-pricing-manipulation/
|   |   +-- <file1>             # ex. poc-pricing.py
|   |   +-- <file2>             # ex. report.md
|   |   +-- ...
|   +-- <turnindir>/            # ex. ex01-race-conditions/
|   |   +-- ...
|   +-- ...
+-- ...
```

- **Répertoire du module** : le nom complet du répertoire tel qu'affiché dans votre curriculum (ex. `M-BLS-001-business-logic`).
- **Répertoire de rendu** : la valeur de `\turnindir` fournie dans chaque exercice (ex. `ex00-pricing-manipulation`).
- **Fichiers de l'exercice** : exactement les fichiers listés dans `\exfiles`. Ni fichiers supplémentaires, ni fichiers manquants.

## II.2 Qu'est-ce qu'une Preuve de Concept Valide

Une PoC doit être :

1. **Reproductible**. N'importe qui avec le même environnement peut l'exécuter et observer la vulnérabilité. Les chemins codés en dur, les adresses IP absolues et les jetons expirés ne sont pas acceptables.

2. **Documentée**. Chaque étape est expliquée. L'évaluteur ne devrait pas avoir à deviner ce que fait le script.

3. **Ciblée**. L'exercice spécifie une cible autorisée (lab, maquette, binaire local ou plateforme désignée). Ne testez jamais les PoC sur des systèmes de production qui ne vous appartiennent pas.

4. **Minimale**. La PoC démontre *uniquement* la vulnérabilité en question. Les charges utiles non liées et le bruit sont pénalisés.

① Une PoC qui modifie silencieusement l'état du système, supprime des fichiers ou persiste des changements en dehors du périmètre de l'exercice sera rejetée. Si votre exploit nécessite un nettoyage, documentez les étapes de nettoyage dans votre rapport.

## II.3 Processus de Soumission

1. Réalisez les livrables de l'exercice sur votre machine locale.

2. Committez et poussez vers votre dépôt Git assigné.

3. Le dépôt est gelé après la date limite.

4. Les pairs clonent votre dépôt et évaluent en utilisant :

   - `pendragon` pour l'exécution automatisée des tests (là où le module fournit des configurations mouli).
   - La grille manuelle pour la qualité du rapport, le style de code et la méthodologie.



Exécutez toujours `pendragon run <MODULE-ID> <ex_id> <path/to/student/repo>` avant votre commit final pour vérifier que votre soumission passe les vérifications automatisées.

### Test avec pendragon

Si le module inclut une configuration mouli (fichiers JSON dans `resources/mouli/`), vous pouvez tester votre soumission localement :

```
pendragon run M-BLS-001 ex00 ~/student-repos/tedsig42
```

`pendragon` résout le chemin de l'exercice automatiquement en cherchant `<mod-fullname>/<ex-dirname>/` dans le répertoire étudiant donné.

### Vérification de style avec caliban

Pour les modules qui nécessitent du code C, `caliban` vérifie la conformité du style :

```
caliban check ~/student-repos/tedsig42/M-BLS-001-business-logic
```

`caliban` applique la Norme 42 (taille des fonctions, nommage des variables, constructions interdites) adaptée aux contextes de recherche de vulnérabilités. Exécutez-le avant la soumission — un échec aux vérifications de style peut invalider l'exercice.

## II.4 Règle des Fichiers Supplémentaires Interdits

- Votre dépôt doit contenir **uniquement** les fichiers listés dans `\exfiles` pour chaque exercice.
- Fichiers interdits : `.DS_Store`, `__pycache__`, `.venv/`, `node_modules/`, `.git/`, binaires compilés (sauf explicitement listés dans `exfiles`), fichiers d'échange d'éditeur, fichiers journaux et tout fichier non demandé.
- Utilisez `.gitignore` pour exclure automatiquement les fichiers indésirables.

## II.5 Format du Rapport

Lorsqu'un rapport (`report.md`) est requis, il doit inclure :

1. **Classe de vulnérabilité** (identifiant CWE si applicable).
2. **Étapes de reproduction** — numérotées, prêtes à être copiées-collées.
3. **Charges utiles** — chaînes exactes, requêtes ou entrées utilisées.
4. **Impact** — ce qu'un attaquant peut accomplir.
5. **Mitigation** — comment le développeur devrait corriger.
6. **Sources** — toutes les ressources consultées (voir Section II.7).

### En-tête de Rendu

Chaque fichier source doit inclure l'en-tête de rendu défini dans `../../../resources/latex/deliverable-header.tex`, adapté à la syntaxe de commentaire du langage utilisé.

## II.6 Cibles Autorisées Uniquement

① Vous pouvez **uniquement** tester les cibles explicitement listées dans l'exercice (labs, serveurs locaux, maquettes, binaires désignés). Tester des systèmes de production réels, des applications tierces ou toute cible qui ne vous appartient pas ou pour laquelle vous n'avez pas d'autorisation écrite est interdit et peut entraîner une disqualification immédiate.

L'éthique du bug bounty s'applique tout au long de ce curriculum :

- Aucun test sur des programmes en production sans un accord de périmètre valide.
- Aucun déni de service sauf si l'exercice l'exige explicitement.
- Aucune exfiltration de données au-delà de ce qui est nécessaire pour démontrer la vulnérabilité.
- Si vous accédez accidentellement à des données réelles d'utilisateurs, arrêtez immédiatement et documentez ce qui s'est passé.

## II.7 Citez Toutes Vos Sources

Toute ressource consultée pendant votre travail doit être citée. Cela inclut :

- Articles académiques et billets de blog.
- Rapports publics de bug bounty (HackerOne, Bugcrowd, etc.).
- Documentation officielle (RFC, pages de manuel, documentation fournisseur).
- **Outils d'IA** : nom du modèle, résumé de la question, date et méthode de vérification (voir le chapitre Instructions IA pour le format de citation exact).
- Extraits de code adaptés de projets open-source (avec attribution de licence si requis).

### Format de citation

```
# R\'ef\'erences

- [1] PortSwigger, "Business Logic Flaws",
  https://portswigger.net/web-security/logic-flaws
  (consult\'e le 2025-03-15).
- [2] Rapport HackerOne #123456, "IDOR on /api/users",
  anonymis\'e, consult\'e via divulgation publique.
- [3] G\'en\'er\'e avec l'assistance de Claude 3.5 Sonnet
  le 2025-03-14. Question : "Write a Python script that...".
  V\'erifi\'e par : revue manuelle de chaque requ\^ete HTTP.
```

## Attribution en ligne

Si un exercice vous demande explicitement d'utiliser une technique ou un outil spécifique, vous n'avez pas besoin de citer l'exercice lui-même comme source. Citez tout ce qui se trouve en dehors de l'énoncé de l'exercice.

# Chapter III

# Instructions sur l'IA

## III.1 Aperçu de la Politique

Les outils de codage assistés par IA (ChatGPT, Claude, Copilot, Gemini, Cursor, Codeium, et similaires) sont **autorisés** tout au long de ce cursus. La recherche en sécurité est un domaine où la maîtrise de l'IA devient essentielle — apprendre à utiliser ces outils efficacement fait partie de votre formation.

Cependant, l'objectif est la **compréhension**, pas l'automatisation. Si vous ne pouvez pas expliquer chaque ligne de votre exploit sans l'aide de l'IA, vous n'avez pas terminé l'exercice.

① Utiliser l'IA pour contourner l'objectif pédagogique d'un exercice — par exemple, lui demander de rétro-ingéniérer un binaire pour ne pas avoir à apprendre le désassembleur, ou de générer une chaîne d'exploitation complète que vous ne comprenez pas — va à l'encontre du but du cursus et sera traité comme une fraude académique.

## III.2 Utilisations Autorisées

- **Explication de concepts**. Demander à l'IA de clarifier une classe de vulnérabilité, un sous-système du noyau, ou un protocole réseau.
- **Échafaudage de code**. Générer du code de départ pour un harnais, un fuzzer, ou un squelette de PoC — puis le modifier et l'étendre vous-même.
- **Débogage**. Expliquer des messages d'erreur, la sortie d'un désassembleur, ou un comportement de crash inattendu.
- **Résumé de documentation**. Condenser une longue RFC ou un article technique en notes exploitables.
- **Traduction**. Traduire du contenu technique entre l'anglais et le français (les deux langues sont utilisées dans ce cursus).

- **Rédaction de rapports**. Demander de l'aide pour structurer un rapport de bug bounty — mais remplir le contenu technique vous-même.

## III.3 Utilisations Interdites

- **Génération complète d'exploit**. Demander à l'IA d'écrire l'exploit complet d'un exercice, puis le soumettre sans le comprendre.
- **Contournement des contraintes de l'exercice**. Utiliser l'IA pour contourner une restriction que l'exercice impose explicitement (p. ex. « n'utilisez pas de scanners automatisés »).
- **Découverte automatisée de vulnérabilités**. Exécuter des outils de scan pilotés par IA qui remplacent l'analyse manuelle que vous êtes censé pratiquer.
- **Rédaction des grilles d'évaluation**. Vous n'êtes pas autorisé à utiliser l'IA pour générer des critères d'évaluation par les pairs ou des réponses.
- **Plagiat sans attribution**. Copier le résultat de l'IA sans le citer (voir Section III.5).

## III.4 Responsabilité Personnelle

Lorsque vous demandez à l'IA de générer du code d'exploit, vous devenez responsable de chaque octet de ce résultat avant de l'utiliser contre une cible.

1. **Lisez le code généré** ligne par ligne.
2. **Identifiez toutes les hypothèses** faites par l'IA (architecture, version du noyau, drapeaux du compilateur, topologie réseau).
3. **Testez en isolation** avant d'exécuter contre une cible d'exercice.
4. **Comprenez les modes d'échec**. Si l'exploit généré plante ou se bloque, vous devez diagnostiquer et corriger le problème vous-même.



Si l'IA génère du code que vous ne comprenez pas, demandez-lui de l'expliquer. Si vous ne comprenez toujours pas après l'explication, le code n'est pas prêt à être utilisé. Passez à une approche plus simple que vous maîtrisez entièrement.

## III.5 Citation Obligatoire de l'IA

Chaque fichier contenant du code généré ou assisté par l'IA doit inclure un commentaire de citation. Le format est :

```
# Generated with assistance from <tool> on <date>.
# Prompt: <one-line summary of the prompt>
# Verified by: <method used to verify the output>
```

### Exemples

**Python** :

```
# Generated with assistance from Claude 3.5 Sonnet on 2025-03-14.
# Prompt: "Write a Python script that sends HTTP requests with
# modified price fields to /api/checkout/pay"
# Verified by: manual review of each request and response
```

**C** :

```
/* Generated with assistance from GitHub Copilot on 2025-03-14.
 * Prompt: "Implement a stack-based buffer overflow with
 * ret2libc for x86_64 Linux"
 * Verified by: compiled with -fno-stack-protector and
 * tested against local binary with ASLR disabled */
```

**Markdown (rapport)** :

```
<!--
Generated with assistance from ChatGPT-4 on 2025-03-14.
Prompt: "Structure a bug bounty report for IDOR with
sections: summary, steps, impact, mitigation"
Verified by: all technical content written manually,
AI only suggested structure
-->
```

### Interactions multiples avec l'IA

Si vous avez utilisé plusieurs outils d'IA, ou itéré à travers plusieurs invites, citez chaque interaction significative. Un groupe d'invites connexes peut être résumé :

```
<!--
AI assistance used in this file:
- Claude 3.5 Sonnet (2025-03-14): harness skeleton generation
- ChatGPT-4 (2025-03-14): crash triage help
- Copilot (2025-03-15): inline autocomplete for loop logic
All generated code reviewed and modified manually.
-->
```

## III.6 Évaluation par les Pairs et l'IA

Lors de l'évaluation par les pairs, l'évaluateur peut :

- Vous demander d'**expliquer tout code généré par l'IA** ligne par ligne.

- Vous demander de **reproduire le raisonnement** sans l'aide de l'IA.
- Demander à voir votre **historique d'invites** pour un exercice donné.
- Déduire des points si la citation de l'IA est manquante ou si vous ne pouvez pas expliquer le code que l'IA a généré.

① Présenter le code généré par l'IA comme votre propre travail (défaut de citation) est du plagiat. Si l'évaluateur soupçonne du plagiat, il peut saisir l'équipe pédagogique pour examen.

## III.7 Rappel Éthique

- Les modèles d'IA peuvent produire du code incorrect, dangereux, ou illégal. Vous êtes **responsable** de vérifier chaque résultat avant exécution.
- Les modèles d'IA entraînés sur la recherche publique en sécurité peuvent reproduire textuellement du code provenant de projets sous licence GPL. Vérifiez les licences avant d'intégrer ces éléments dans votre travail.
- Ne saisissez pas de code sensible ou propriétaire dans des outils d'IA hébergés dans le cloud. Utilisez des modèles locaux (Llama, Mistral, DeepSeek) pour les travaux impliquant des cibles non publiques.
- L'éthique de la recherche en sécurité que vous apprenez ici s'applique à l'utilisation de l'IA : ne demandez pas à l'IA de générer des malwares, des rançongiciels, ou du code destiné à nuire à des systèmes en dehors du cadre de l'exercice.



La maîtrise des outils d'IA est une compétence recherchée dans la chasse aux bogues moderne. Utilisez-les sagement, citez-les honnêtement, et conservez toujours votre propre compréhension de chaque exploit que vous déployez.

# Chapter IV

# Avant-propos

La chaîne d'authentification Windows n'est pas un seul composant. C'est une machine à états répartie entre le client, le serveur, le contrôleur de domaine et les services qui acceptent des tickets ou des jetons.

Une *réflexion* apparaît quand une authentification destinée à une frontière de confiance est réutilisée sur une autre frontière. Un *contournement de frontière* apparaît quand un composant accepte une preuve qui n'a pas été produite pour lui.

| **Étape** | **Question d'audit** |
|---|---|
| Preuve | Qui a produit cette preuve, pour quelle cible, et pour quelle durée ? |
| Transport | La preuve est-elle liée au canal ou au service qui la reçoit ? |
| Validation | Le vérificateur applique-t-il la même règle que l'émetteur ? |
| Translation | La preuve est-elle transformée en un autre type de jeton sans perte de contexte ? |



L

es CVE de 2026 évoluent. Ne reprenez pas un chiffre ou un mécanisme sans citer l'avis officiel. Le but du module est la méthode, pas la mémorisation d'un exploit.

# Chapter V

# Aperçu du module

| Ex | Titre | Difficulté | Points | |
|---|---|---|---|---|
| 00 | Carte de la machine à états | Moyen | 4 | |
| 01 | CVE-2025-33073 : réflexion NTLM | Difficile | 5 | |
| 02 | CVE-2025-58726 : Ghost SPNs | Difficile | 4 | |
| 03 | 2026 : chaîne et correctif incomplet | Difficile | 5 | |
| 04 | Laboratoire et détection | Moyen | 4 | |
| | **Total** | | **22** | |

# Chapter VI

# Exercise 00

|  | Exercise 00 |
|---|---|
| Carte de la machine à états | |
| Turn-in directory : `ex00/` | |
| Files to turn in : `fsm.md` | |
| Allowed functions : `un éditeur de texte, documentation Microsoft` | |

Dans `fsm.md`, modélisez une authentification NTLM/Kerberos simplifiée :

1. les acteurs : client, serveur, KDC, service ;
2. les messages échangés ;
3. les preuves produites et leur cible ;
4. les transitions d'état ;
5. les points où une preuve est traduite ou réutilisée.

Ajoutez trois colonnes : *qui émet*, *qui vérifie*, *quelle frontière est franchie*.



Une machine à états d'authentification est un Logical Engine : le bug apparaît souvent dans une transition d'erreur ou de repli, pas dans le chemin nominal.

# Chapter VII

# Exercise 01

|  | Exercise 01 |
|---|---|
| CVE-2025-33073 : réflexion NTLM | |
| Turn-in directory : `ex01/` | |
| Files to turn in : `cve-2025-33073.md` | |
| Allowed functions : `un éditeur de texte, NVD, avis Synacktiv` | |

CVE-2025-33073 est décrite comme une élévation de privilèges dans le client SMB Windows, liée à une réflexion d'authentification NTLM. Le cas est intéressant parce que la preuve est valide, mais elle est utilisée sur une mauvaise frontière.

## Mission

Dans `cve-2025-33073.md` :

1. décrivez le chemin de réflexion sans fournir de commande d'exploitation ;
2. expliquez pourquoi le serveur accepte une preuve produite pour le client ;
3. identifiez l'hypothèse de liaison au canal qui manque ;
4. proposez une détection défensive (journal, événement, indicateur) ;
5. expliquez pourquoi un correctif local sur un seul composant peut être insuffisant.

## Sources

- NVD : https://nvd.nist.gov/vuln/detail/CVE-2025-33073
- Synacktiv, analyse de CVE-2025-33073.
- Documentation Microsoft sur la liaison de canal NTLM.

# Chapter VIII

# Exercise 02

|  | Exercise 02 |
|---|---|
| CVE-2025-58726 : Ghost SPNs | |
| Turn-in directory : `ex02/` | |
| Files to turn in : `cve-2025-58726.md` | |
| Allowed functions : `un éditeur de texte, NVD, avis Semperis` | |

CVE-2025-58726 est associée à des « Ghost SPNs » et à une confusion de service dans l'authentification Windows. Le mécanisme exact doit être vérifié dans les avis publics.

## Mission

Dans `cve-2025-58726.md` :

1. définissez ce qu'est un SPN et comment il est utilisé pour choisir un service ;
2. expliquez le risque d'un SPN fantôme ou mal enregistré ;
3. reliez le cas à la question de frontière de confiance ;
4. proposez un contrôle de configuration qui réduit l'exposition ;
5. listez les affirmations que vous n'avez pas pu vérifier.

## Sources

- NVD : https://nvd.nist.gov/vuln/detail/CVE-2025-58726
- Semperis, analyse des Ghost SPNs.

# Chapter IX

# Exercise 03

|  | Exercise 03 |
|---|---|
| 2026 : chaîne et correctif incomplet | |
| Turn-in directory : `ex03/` | |
| Files to turn in : `chain.md` | |
| Allowed functions : `un éditeur de texte, NVD, avis éditeurs` | |

Étudiez quatre CVE de 2026 :

- CVE-2026-24294 : port SMB arbitraire ;
- CVE-2026-26128 : homoglyph DNS ;
- CVE-2026-26119 : contournement EPA dans Windows Admin Center ;
- CVE-2026-50343 : Dark Elevator, chargement d'une DLL plantable par `InstallService`.

Dark Elevator combine deux défauts logiques : une carte de plugins inscriptible par un utilisateur normal, et une classe COM Windows dont la DLL in-process est enregistrée sous `%PROGRAMDATA%`, donc plantable. Le service SYSTEM charge la DLL dans son propre processus. Le suivi Project Zero, CVE-2026-66804, montre que le correctif de `InstallService` laisse l'enregistrement COM orphelin.

## Mission

Dans `chain.md`, remplissez un tableau :

| **CVE** | **Hypothèse du correctif** | **Frontière contournée** |
|---|---|---|
| CVE-2026-24294 | | |
| CVE-2026-26128 | | |
| CVE-2026-26119 | | |
| CVE-2026-50343 | | |

Puis répondez :

1. Ces quatre cas sont-ils des variantes d'une même racine, ou des bugs indépendants qui partagent un thème ?

2. Quel contrôle défensif couvre plusieurs cas à la fois ?

3. Pourquoi la validation d'un nom ou d'un port ne suffit-elle pas à garantir la frontière ?

4. Pourquoi le correctif de `InstallService` ne ferme-t-il pas l'enregistrement COM orphelin de CVE-2026-66804 ?

5. Classez CVE-2026-66804 : *incomplete-fix variant* ou *architectural scatter* ? Justifiez avec la définition du papier.

6. Proposez trois contrôles défensifs : ACL de la carte de plugins, inventaire des enregistrements COM, surveillance des DLL créées sous `ProgramData`.



L

e suivi Project Zero montre qu'un correctif peut fermer le chemin d'activation sans supprimer la ressource dangereuse. Ne reproduisez pas la chaîne de chargement ; travaillez sur la carte des frontières et les contrôles de configuration.

# Chapter X

# Exercise 04

|  | Exercise 04 |
|---|---|
| Laboratoire et détection | |
| Turn-in directory : `ex04/` | |
| Files to turn in : `lab.md` | |
| Allowed functions : `un éditeur de texte, documentation AD, outils de journalisation` | |

Concevez un laboratoire défensif, pas un terrain d'attaque :

1. un domaine AD simulé avec des comptes jetables ;
2. un serveur SMB isolé ;
3. un client Windows dans un réseau séparé ;
4. des journaux centralisés ;
5. des règles de détection pour les réflexions et les SPN suspects.

Dans `lab.md`, décrivez :

- le schéma réseau et les frontières ;
- les événements à collecter ;
- trois requêtes de détection ;
- les faux positifs attendus ;
- les critères d'arrêt du laboratoire.



N

e testez jamais ces techniques sur un domaine qui ne vous appartient pas. Une preuve d'autorisation écrite est obligatoire avant toute manipulation.

# Chapter XI

# Rendu et évaluation

```
A-EPI-004-windows-auth-chain/
|-- ex00/fsm.md
|-- ex01/cve-2025-33073.md
|-- ex02/cve-2025-58726.md
|-- ex03/chain.md
+-- ex04/lab.md
```

| **Critère** | **Attendu** | **Points** |
|---|---|---|
| FSM | Acteurs, preuves et frontières explicites | 5 |
| Réflexion | Mécanisme et liaison de canal | 5 |
| SPN | Rôle du SPN et contrôle défensif | 4 |
| Chaîne 2026 | Dark Elevator, correctif incomplet, sources | 4 |
| Laboratoire | Isolé, détections et critères d'arrêt | 4 |

# Chapter XII

# Ressources par exercice

### Exercice 00 — Carte de la machine à états

- Microsoft, NTLM : https://learn.microsoft.com/en-us/windows-server/security/kerberos/ntlm-overview
- Microsoft, Kerberos : https://learn.microsoft.com/en-us/windows-server/security/kerberos/kerberos-authentication-overview
- Microsoft, SMB : https://learn.microsoft.com/en-us/windows-server/storage/file-server/file-server-smb-overview
- Papier, section 16.1.
- À chercher : `NEGOTIATE`, `CHALLENGE`, `AUTHENTICATE`, `AS-REQ`, `TGS-REQ`.

### Exercice 01 — CVE-2025-33073

- NVD : https://nvd.nist.gov/vuln/detail/CVE-2025-33073
- MSRC : https://msrc.microsoft.com/update-guide/vulnerability/CVE-2025-33073
- Synacktiv, analyse NTLM reflection : https://www.synacktiv.com/sites/default/files/2026-05/bypassing_authentication_reflections_mitigations_for_system_shells.pdf
- Papier, section 16.2.
- À chercher : `mrxsmb.sys`, `CMTI`, `SPN`, `channel binding`, `loopback check`.

### Exercice 02 — CVE-2025-58726

- NVD : https://nvd.nist.gov/vuln/detail/CVE-2025-58726
- MSRC : https://msrc.microsoft.com/update-guide/vulnerability/CVE-2025-58726
- Semperis, Ghost SPNs : https://www.semperis.com/blog/exploiting-ghost-spns-and-kerberos-reflection

- À chercher : `Ghost SPN`, `service account`, `KDC`, `ticket`.

## Exercice 03 — Chaîne 2026 et correctif incomplet

- NVD : CVE-2026-24294, CVE-2026-26128, CVE-2026-26119, CVE-2026-50343, CVE-2026-66804.
- MSRC : https://msrc.microsoft.com/update-guide/
- Calif, Dark Elevator : https://blog.calif.io/p/dark-elevator-windows-install-service
- Project Zero, dangling COM : https://projectzero.google/2026/09/windows-dangling-com.html
- Synacktiv, PDF auth reflections (même lien que l'exercice 01).
- À chercher : `InstallService`, `StaticPluginMap`, `CrossDevice`, `ProgramData`, `COM orphan`.

## Exercice 04 — Laboratoire et détection

- Microsoft, AD DS : https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/active-directory-domain-services
- Microsoft, audit events : https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/advanced-security-audit-policy
- Papier, sections 16.2 et 16.3.
- À chercher : `event 4624`, `event 4768`, `event 4769`, `SPN`, `loopback`.

# Chapter XIII

# Sources


- Phrack, *Security is failure: Every Patch Is a Guess*, sections 9 et 10.
- NVD : CVE-2025-33073, CVE-2025-58726, CVE-2026-24294, CVE-2026-26128, CVE-2026-26119, CVE-2026-50343, CVE-2026-66804.
- Calif, *Dark Elevator: Windows Install Service Local Privilege Escalation* : https://blog.calif.io/p/dark-elevator-windows-install-service
- Project Zero, *Dangling COM Object Registrations* : https://projectzero.google/2026/09/windows-dangling-com.html
- Synacktiv, *The Gift That Keeps on Giving* : https://www.synacktiv.com/sites/default/files/2026-05/bypassing_authentication_reflections_mitigations_for_system_shells.pdf
- Documentation Microsoft sur NTLM, Kerberos, SMB, EPA et COM.
- Analyses Semperis.